CSE499B – Section 15 – Group 03
Md. Rahad Hossain, Md. Yousuf, MD. Rokib Hasan Oli, Safayat Ibrahim

CityPulse: Intelligent Parking & Traffic Control Platform

Weekly Progress Report – Update 1 (Sprint 1: Bangla Licence Plate Detection)

I. Sprint Overview

This update covers the first week of CSE499B on the backend/AI, frontend, and mobile branches. The main work was on the AI side. Bangla licence plate recognition is built in two stages: a detector finds the plate in the frame, and a second model reads its text. This week completed stage one: public datasets were collected and merged, plate detectors were trained, and all models were evaluated on common test sets.

II. Datasets

Three public sources with full-frame images and plate boxes were collected: the Bangladeshi License Plate Recognition dataset [1] (6,517 images, Pascal VOC), BLPD [2] (2,140 images, YOLO), and a Roboflow Universe set (2,142 images, YOLO). The plate-crop dataset used in [3] is no longer available on Kaggle, and only the videos of [4] remain public (50 of 79 retrieved); neither is used for detection.

III. Detection Dataset

The sources were merged into one single-class YOLO dataset. Polygon labels were converted to boxes, and vehicle classes were dropped. Images were compared by perceptual hash; 847 training and 20 validation images that duplicated validation or test images were removed, so no test image appears in training. Table I shows the split. Each source keeps its own train/validation/test division, and 17% of images have no plate. In the merged rows the counts are after duplicate removal. The paper [3] uses only the first source with its original, untouched split (last row); ours uses all three. The 553 test images of [1] are identical in both, so results can be compared.

TABLE I
Images per split after duplicate removal (plate boxes: 9,477)

Source	Train	Val	Test
Hossain et al. [1]	5,220	274	553
BLPD [2]	1,386	415	213
Roboflow	1,630	142	99
Ours (merged)	**8,236**	**831**	**865**
Paper setup ([1] only)	5,687	277	553

IV. Training

All models were fine-tuned from COCO-pretrained weights with Ultralytics 8.4.46 on a Tesla T4. Table II lists the settings we changed for every run; all runs use image size 640 and seed 0. The two-stage run follows [3] (Table III). Stage 1 trains for 35 epochs with 12 frozen layers, reduced to 8 at epoch 13 and to 4 at epoch 25. Stage 2 continues from the best stage-1 checkpoint for 45 epochs with all layers unfrozen and a cosine learning-rate schedule. The paper does not give the optimizer, learning rates, momentum, weight decay, or the epochs at which layers are unfrozen; these are our choice. Every setting not listed is the Ultralytics default.

TABLE II
Settings changed from the defaults (all runs)

Model	Data	Epochs	Batch	Early stop
YOLO26n / s / m	Merged	100	32	patience 20
YOLOv8m	Paper	80	10	patience 20
YOLOv8m two-stage	Paper	35 + 45	10	patience 15 (stage 1)

TABLE III
Two-stage settings (default in parentheses)

Setting	Stage 1	Stage 2
Frozen layers (0)	12, 8, 4	0
Rotation (0)	8°	3°
Translation (0.1)	0.15	0.08
Scale (0.5)	0.7	0.4
Shear (0)	3°	1°
Mosaic (1.0)	1.0	0.7
Mix-up (0)	0.15	0.08
Copy-paste (0)	0.4	0.2
Horizontal flip (0.5)	0.5	0.3
HSV (0.015 / 0.7 / 0.4)	0.01 / 0.8 / 0.5	0.02 / 0.9 / 0.7
Optimizer (auto)	SGD	SGD
Learning rate (0.01)	0.01	0.001
Cosine schedule (off)	off	on
Momentum (0.937)	0.95	0.9
Weight decay (0.0005)	0.001	0.0005
Warm-up epochs (3)	3	0

V. Results

TABLE IV
Test results (mAP@0.5 / mAP@0.5:0.95)

Model	Paper test	Full test	ms
YOLO26n	0.991 / 0.728	**0.981** / 0.730	**3.0**
YOLO26s	0.990 / 0.743	0.980 / 0.737	7.7
YOLO26m	0.988 / 0.743	0.978 / **0.739**	23.7
YOLOv8m	0.991 / **0.763**	0.910 / 0.610	20.9
YOLOv8m two-stage	**0.994** / **0.763**	0.912 / 0.587	20.4

In Table IV, *paper test* is the 553-image split of [1] and *full test* covers all sources (865 images). On the paper test

all models reach about 0.99 mAP@0.5, and YOLO26n (2.4M parameters) matches YOLOv8m (25.8M). On the full test the single-dataset models fall to 0.91, while the YOLO26 models stay at 0.98. YOLO26m (20.4M) is no better than YOLO26n.

The two-stage model has the highest paper-test mAP@0.5 (0.994). Its accuracy (all plates of an image found with IoU > 0.7) is 96.93%, against 97.11% for our plain YOLOv8m and 97.83% reported in [3], so the paper's accuracy gain is not reproduced. In stage 1 the loss became unstable after the first unfreezing step and the run stopped early at epoch 34; the checkpoint passed to stage 2 comes from the part with 12 frozen layers.

VI. Key Findings

Training on merged sources matters more than model size: the smallest model generalises as well as the largest across sources. No model is selected yet. A custom detector will be built next and compared with these baselines before the final choice.

VII. Frontend Progress

Frontend work connected the owner portal pages to the backend API in place of placeholder data.

- **Owner dashboard:** metric cards, the list of parking lots with their live slot status, and the camera list now come from the spaces and bookings APIs, with loading and error states.
- **Cameras:** the page lists the owner's lots and cameras from the parking spaces API, shows a status strip and camera grid, and has an add/edit camera form with a local image preview.
- **Revenue history:** bookings load from the bookings API with filters and summary cards; transactions show as a table on desktop and as cards on mobile, and a booking can be cancelled from the page.
- **Facility settings:** general details, location (map picker), camera setup, photos, and pricing load and save through the spaces API. Photos can be uploaded and deleted without losing unsaved edits. Surge pricing and auto-payout are shown as "coming soon" until their APIs exist.
- **Register lot:** the form (location, amenities, opening hours, photos) creates the parking space through the API and then uploads its photos.
- **Paper review:** a Bangla plate reading paper (GFP-GAN restoration, then a CNN over 17 character classes, 92.4% character accuracy). The review found duplicated images between its training and test sets, so the reported accuracy is optimistic.
- 499A screenshots were archived and new screenshots of the five pages were added.

VIII. Mobile Progress

Mobile work was mainly restructuring of the driver app before new features are added.

- **Dashboard refactor:** the 1,452-line dashboard screen was reduced to 145 lines. Its parts became 13 separate widgets (map, search bar and suggestions, parking card and strip, details sheet, route card, booking confirmation and exit dialogs), with shared map and marker utilities.
- **API service:** the nearby-parking request was moved out of the screen into its own service class with a timeout and cancellation of the previous request.
- **Theme:** dark mode was removed from the dashboard widgets so the screen has one consistent look.
- **Paper review:** the detection stage of a YOLOv8 Bangla plate recognition paper was reviewed and re-run in a Kaggle notebook. YOLOv8n was fine-tuned from COCO weights on the paper's Roboflow dataset (10,412 training and 2,970 validation images) for 30 epochs at image size 640, batch 32, on two T4 GPUs with mixed precision; all other settings are the Ultralytics defaults. Training took about one hour and reached 0.988 mAP@0.5 on the validation set (Table V). The trained weights and sample detections on test images were saved with the notebook.

TABLE V
YOLOv8n plate detection, validation results (2,968 images)

Model	Precision	Recall	mAP@0.5	mAP@0.5:0.95
YOLOv8n	0.971	0.980	0.988	0.784

- **Documentation:** the mobile README was rewritten with a feature status table, the app architecture and navigation flow, the backend API used by the app, setup steps, and a roadmap for 499B.

IX. Plan for Next Week

- Frontend: complete the API integration of the owner pages and test them against the live API.
- Mobile: connect the remaining screens to the backend API and add the plate reading stage to the paper reproduction.
- Design and train a custom plate detector and compare it with these baselines.
- Build the character dataset from the plate crops of [1] and [2].
- Continue Dhaka parking footage collection.
- Start the RTSP stream worker.

References

[1] S. N. Hossain and M. Z. Hassan, "Bangladeshi license plate recognition dataset," Kaggle, 2023.

[2] P. Sarker and S. M. S. Alam, "Cascaded YOLO-based approach for Bangladeshi license plate identification and number recognition," *PLOS ONE*, vol. 21, no. 8, 2026.

[3] N. Hasin *et al.*, "A robust deep learning framework for Bangla license plate recognition using YOLO and vision-language OCR," *arXiv:2603.10267*, 2026.

[4] A. Ashrafee *et al.*, "Real-time Bangla license plate recognition system for low resource video-based applications," *arXiv:2108.08339*, 2021.